Nama: Hans Malvin Djojo

https://github.com/hansmalvin/LastMile

Environment Setup
mongosh
use salesData
create a sales.json:


```
[
    {
     "_id": 1,
     "customerId": "A123",
     "product": "Laptop",
     "quantity": 1,
     "price": 1200,
     "date": "2023-01-15"
    },
    {
     "_id": 2,
     "customerId": "B456",
     "product": "Mouse",
     "quantity": 2,
     "price": 25,
     "date": "2023-01-20"
    },
    {
     "_id": 3,
     "customerId": "A123",
     "product": "Keyboard",
     "quantity": 1,
     "price": 75,
     "date": "2023-02-01"
    },
    {
     "_id": 4,
     "customerId": "C789",
     "product": "Laptop",
     "quantity": 1,
     "price": 1300,
     "date": "2023-02-10"
    },
    {
     "_id": 5,
     "customerId": "B456",
     "product": "Monitor",
     "quantity": 1,
     "price": 300,
     "date": "2023-02-15"
    },
    {
     "_id": 6,
     "customerId": "A123",
```

```
    "product": "Webcam",
    "quantity": 1,
    "price": 50,
    "date": "2023-03-01"
  },
  {
    "_id": 7,
    "customerId": "D012",
    "product": "Headphones",
    "quantity": 1,
    "price": 150,
    "date": "2023-03-10"
  },
  {
    "_id": 8,
    "customerId": "C789",
    "product": "Keyboard",
    "quantity": 1,
    "price": 80,
    "date": "2023-03-15"
  },
  {
    "_id": 9,
    "customerId": "E345",
    "product": "Mouse",
    "quantity": 2,
    "price": 30,
    "date": "2023-03-20"
  },
  {
    "_id": 10,
    "customerId": "A123",
    "product": "Laptop",
    "quantity": 1,
    "price": 1250,
    "date": "2023-04-01"
  }
]
```


```
mongoimport --db salesData --collection transactions --file sales.json --jsonArray
db.transactions.find().limit(5)
```

Step 1: Grouping by Customer and Calculating Total Spending

```
db.transactions.aggregate([
 {
  $group: {
    _id: "$customerId",
    totalSpending: { $sum: { $multiply: ["$quantity", "$price"] } }
  }
 }
])
```

Explanation:

The $group stage groups documents by the customerId field. The totalSpending field is calculated by multiplying the quantity and price fields for each transaction and summing the results using the $sum operator. The $multiply operator is used within $sum to calculate the spending per transaction.

```
salesData> db.transactions.aggregate([
|    {
|      $group: {
|        _id: "$customerId",
|        totalSpending: { $sum: { $multiply: ["$quantity", "$price"] } }
|      }
|    }
| ])
|
[
  { _id: 'B456', totalSpending: 350 },
  { _id: 'A123', totalSpending: 2575 },
  { _id: 'C789', totalSpending: 1380 },
  { _id: 'E345', totalSpending: 60 },
  { _id: 'D012', totalSpending: 150 }
]
salesData>
```

Step 2: Filtering Customers with Spending Greater Than a Threshold

```
db.transactions.aggregate([
 {
  $group: {
   _id: "$customerId",
   totalSpending: { $sum: { $multiply: ["$quantity", "$price"] } }
  }
 },
 {
  $match: {
   totalSpending: { $gt: 500 }
  }
 }
])
```

Explanation:

The $match stage filters the results from the previous $group stage, selecting only those documents where the totalSpending field is greater than $500. This allows us to focus on high-value customers.

```
salesData> db.transactions.aggregate([
|   {
|     $group: {
|       _id: "$customerId",
|       totalSpending: { $sum: { $multiply: ["$quantity", "$price"] } }
|     }
|   }
| ])
|
[
  { _id: 'B456', totalSpending: 350 },
  { _id: 'A123', totalSpending: 2575 },
  { _id: 'C789', totalSpending: 1380 },
  { _id: 'E345', totalSpending: 60 },
  { _id: 'D012', totalSpending: 150 }
]
salesData> db.transactions.aggregate([
|   {
|     $group: {
|       _id: "$customerId",
|       totalSpending: { $sum: { $multiply: ["$quantity", "$price"] } }
|     }
|   },
|   {
|     $match: {
|       totalSpending: { $gt: 500 }
|     }
|   }
| ])
|
[
  { _id: 'C789', totalSpending: 1380 },
  { _id: 'A123', totalSpending: 2575 }
]
salesData>
```

Step 3: Projecting Relevant Fields and Renaming

```
db.transactions.aggregate([
 {
  $group: {
   _id: "$customerId",
   totalSpending: { $sum: { $multiply: ["$quantity", "$price"] } }
  }
 },
 {
  $match: {
   totalSpending: { $gt: 500 }
  }
 },
 {
  $project: {
   _id: 0,
   customerId: "$_id",
   total: "$totalSpending"
  }
 }
])
```

Explanation:

The $project stage reshapes the documents. _id: 0 excludes the _id field. The customerId field is created by assigning the value of the _id field from the previous stage ($_id). The total field is created by assigning the value of the totalSpending field.

```
salesData> db.transactions.aggregate([
|   {
|     $group: {
|       _id: "$customerId",
|       totalSpending: { $sum: { $multiply: ["$quantity", "$price"] } }
|     }
|   },
|   {
|     $match: {
|       totalSpending: { $gt: 500 }
|     }
|   }
| ])
|
[
  { _id: 'C789', totalSpending: 1380 },
  { _id: 'A123', totalSpending: 2575 }
]
salesData> db.transactions.aggregate([
|   {
|     $group: {
|       _id: "$customerId",
|       totalSpending: { $sum: { $multiply: ["$quantity", "$price"] } }
|     }
|   },
|   {
|     $match: {
|       totalSpending: { $gt: 500 }
|     }
|   },
|   {
|     $project: {
|       _id: 0,
|       customerId: "$_id",
|       total: "$totalSpending"
|     }
|   }
| ])
|
[
  { customerId: 'C789', total: 1380 },
  { customerId: 'A123', total: 2575 }
]
salesData>
```

Step 4: Unwinding an Array (Hypothetical Scenario)

```
db.transactions.updateMany(
 {},
 [
  {
   $set: {
    categories: {
     $cond: {
      if: { $gt: ["$price", 500] },
      then: ["expensive", "electronics"],
      else: ["cheap", "accessories"]
```

```
        }
       }
      }
    }
  ]
)

db.transactions.aggregate([
  {
    $unwind: "$categories"
  },
  {
    $group: {
      _id: "$categories",
      count: { $sum: 1 }
    }
  }
])
```

Explanation:

The $unwind stage deconstructs the categories array field from the input documents to output a document for each element in the array. Then we group by category and count the occurences. The updateMany command adds a categories array to each document based on whether the price is greater than 500. The $cond operator is used to conditionally set the array values. This allows us to practice the $unwind stage even with our initial dataset structure.

Step 5: Sorting Results

```
db.transactions.aggregate([
 {
  $group: {
   _id: "$customerId",
   totalSpending: { $sum: { $multiply: ["$quantity", "$price"] } }
  }
 },
 {
  $match: {
   totalSpending: { $gt: 500 }
  }
 },
 {
```

```
  $project: {
    _id: 0,
    customerId: "$_id",
    total: "$totalSpending"
  }
 },
 {
  $sort: {
    total: -1
  }
 }
])
```

Explanation:

The $sort stage sorts the documents based on the total field in descending order (indicated by -1). Sorting allows for easier analysis and presentation of the results. In this case, the output is already sorted due to the small dataset and previous filtering, but this step demonstrates how to explicitly sort the output.